

AI Systems Performance Engineering

Chapter 6 — GPU Architecture, CUDA Programming, and Maximizing Occupancy Part I structure preview (draft)

Tianqin Cai
Applied Scientist
AWS

Internal Training for Applied Scientists on AI Infrastructure

August 15, 2026

What This Chapter Covers

- ① **SIMT execution model** — how **warps**, **thread blocks**, and **grids** map your algorithm onto **SMs**. (Part I)
- ② **CUDA programming patterns** — kernel anatomy, launch parameters, asynchronous allocation. (Part II)
- ③ **Memory hierarchy** — register file, shared/L1, L2, HBM3e, plus asynchronous data movement (**TMA**, **TMEM**). (Part III)
- ④ **Roofline analysis** — is a kernel **compute-bound** or **memory-bound**? Push toward **peak throughput**. (Parts IV–V)

Where does occupancy fit?

Occupancy is the working metric that connects the SIMT model (Part I) to peak throughput (Parts IV–V): enough **resident warps** keep SMs busy while data moves.

*Plain English: the arc: **how the GPU executes** → how you program it → where data lives → **what limits speed**.*

Roadmap: Five Parts, Each Answers One Question

Part	Slides	The question it answers
I GPU architecture	4–20	How does SIMT map warps/blocks/grids onto SMs?
II CUDA programming	21–28	How do we map a computation onto GPU threads ?
III Memory hierarchy	29–37	Where does data live , and what does movement cost ?
IV Occupancy in practice	38–46	When do more warps actually help?
V Correctness & roofline	47–50	Optimize compute next, or memory ?

The whole chapter in one sentence

Expose enough parallelism, keep data close, and profile the real bottleneck.

When a detail page appears (SFU, TMEM, Unified Memory...), it always belongs to one of these five questions.

Part I — GPUs Optimize Throughput; CPUs Optimize Latency

- CPUs: few cores, deep caches, fast *single* threads. GPUs: hundreds of **SMs** (streaming multiprocessors) running **thousands of threads** in parallel.
- Basic CUDA flow (right): host copies data **CPU → GPU**, launches a **kernel**, copies results back.
- The GPU's edge is **massive parallelism**: many SMs process different data **at once**; within an SM, **resident warps** also fill memory stalls.
- Sweet spot: **data-parallel** work — matrix multiplies, convolutions — written in CUDA C++ or via **PyTorch** / OpenAI **Triton**.^[1,3]

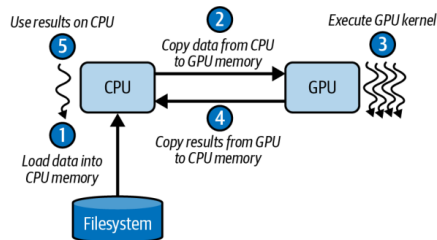


Figure 6-1. Simple CUDA programming flow

*Plain English: the CPU finishes one part fast; the GPU is a **factory** — keep **every workshop** busy.*

Inside a Blackwell SM: The Resource Budget

- Each SM tracks up to **64 warps** (**warp** = 32 threads; details p. 13) = **2,048 threads** — the pool the scheduler switches between.
- **64K 32-bit registers** per SM (256 KB total); any single thread can use at most **255**.
- **256 KB** combined L1 cache / shared memory; up to **228 KB** configurable as user-managed shared memory (**227 KB usable** — CUDA reserves 1 KB per block).
- These on-chip budgets let one SM juggle thousands of threads without constantly touching DRAM.^[2]

Plain English: an SM is a workshop: registers are toolbelts, shared memory is the workbench.

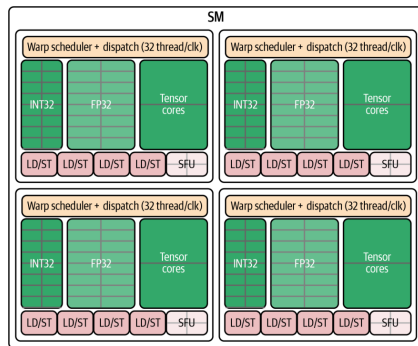


Figure 6-2. A Blackwell SM: four scheduling partitions sharing on-chip resources

SFUs and Load/Store Pipelines

SFU — Special Function Unit

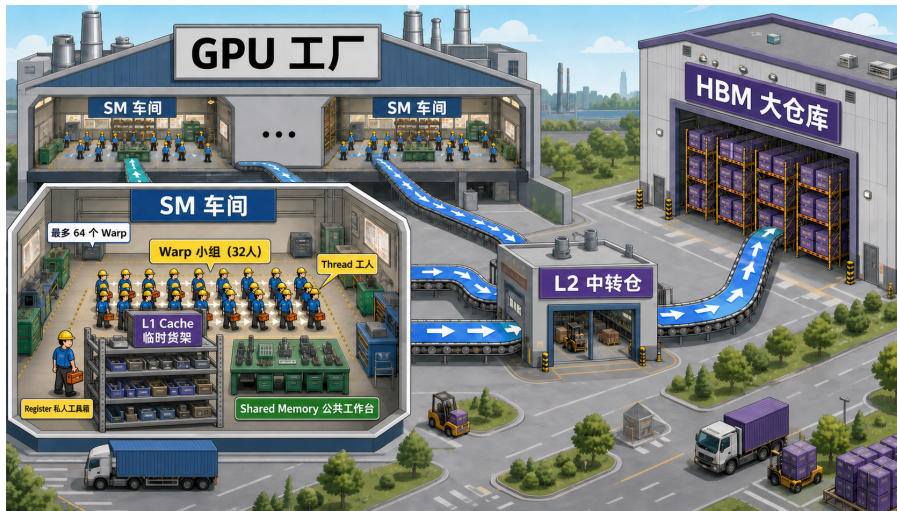
- Handles **transcendentals**: sine, cosine, reciprocal, square root.
- Own **dedicated pipeline** — *not* part of the dual-issue math+memory pair (p. 8).
- Slow functions never stall the core compute/memory pipes $\Rightarrow$ more **instruction-level parallelism** for mixed-operation kernels.

LD/ST — Load/Store pipelines

- **16 LD/ST pipelines** per SM (4 per scheduler) feed L1/shared, L2, and global memory.
- **Caveat (book warning)**: exact counts and pairings are **not guaranteed** — rely on profiling counters and the **Blackwell tuning guide**.^[2]

Plain English: the SFU is a specialist counter at the back of the store — complicated requests go there so the express lanes keep moving.

Analogy: The GPU as a Factory



SM = workshop L2 = central depot HBM = warehouse (presenter's figure)

Four Warp Schedulers, Dual Issue

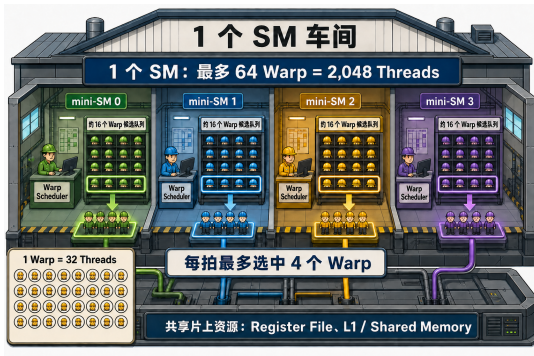
- Each SM = four “**mini-SMs**”: independent warp schedulers with their own dispatch logic.
- Each scheduler issues **1 warp instruction per cycle** $\Rightarrow$ up to **4 warps advance per cycle**.
- **Dual-issue**: one math (INT32/FP32/Tensor Core) **+** one memory (load/store) instruction per cycle — *from the same warp*, never across warps.
- Best case: **4 math + 4 memory** instructions in flight every cycle.

*Plain English: **four dispatchers** on one floor, each sending out one crew per beat — and a crew can compute with one hand while fetching parts with the other.*

Per cycle (Table 6-1)	Limit
Schedulers	4
Warps issued	4
Math / memory ops	4 / 4

Book note: table values are illustrative; real benchmarks in the GitHub repo^[7]

Analogy: One SM Is a Workshop



Presenter's analogy figure (not from the book)

- Four bays = **4 scheduling partitions** ("mini-SMs"), one **warp scheduler** each — CUDA only ever sees the whole SM.
- Hard hats on shelves = **resident warps**: ~16 per partition, **64 total**. Resident = state already allocated — *not* "executing right now."
- Highlighted crews = the **≤ 4 warps issued this cycle** (each scheduler picks at most 1 ready warp).
- Bottom left: **1 warp = 32 threads** — dispatched as a whole crew.
- Common foundation: **register file, L1, shared memory** — shared by all four partitions.

Plain English: 64 crews on standby, 4 dispatchers, at most 4 crews per beat — each crew is 32 workers.

The Thread Hierarchy: Threads → Blocks → Grids

- **Thread**: runs your kernel code on one data element.
- **Thread block** (CTA — cooperative thread array): up to **1,024 threads**; shares fast on-chip **shared memory**.
- **Grid**: all blocks of one launch — sized right, it scales to **millions of threads** with zero kernel changes.
- The CUDA runtime (and PyTorch) handles scheduling and distribution across all SMs.

Plain English: workers (threads) form teams (blocks) that talk cheaply inside the team; the company (grid) hires as many teams as the job needs.

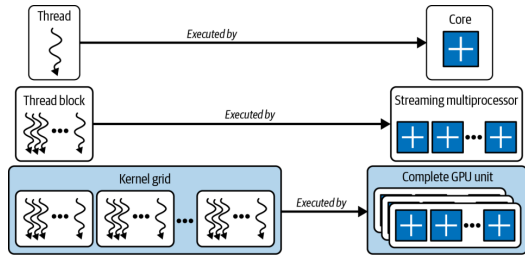


Figure 6-3. Threads, thread blocks (CTAs), and grids

Blocks Cooperate Inside, Stay Independent Outside

- Within a block: share data via shared memory, synchronize with `__syncthreads()` (a barrier: everyone arrives before anyone continues).
- **Every barrier costs time** — the book's advice: **minimize synchronization points**.
- Across blocks: **independent, no guaranteed order**. That freedom lets the scheduler spray blocks over all SMs — and lets your kernel run **unmodified on future GPUs** with more SMs.

Plain English: team huddles (barriers) are useful but expensive — call as few as possible; and never assume team A finishes before team B.

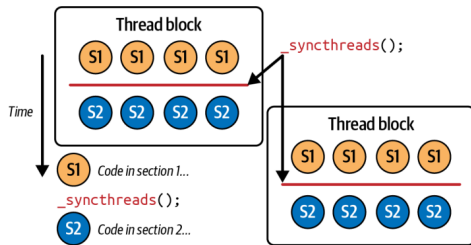


Figure 6-6. `__syncthreads()` between two code sections

Thread Block Clusters and DSMEM

* *Good to know for LLM-scale matmuls — not the focus of this session (details: Chapter 10).*

- Traditionally, threads in *different* blocks could not cooperate. Modern GPUs add **thread block clusters**: blocks that communicate **across SMs**, with cluster-scoped hardware barriers.
- Backed by **DSMEM** (distributed shared memory): the shared-memory banks of participating SMs, linked over a fast **on-chip interconnect** into one pool.
- Threads read, write, and atomically update each other's shared buffers **at on-chip speeds** — **without global-memory bandwidth**.
- Key enabler for huge matmuls in LLMs — details in **Chapter 10**.

Plain English: neighboring teams knock a door through the wall between their workbenches instead of mailing parts through the warehouse.

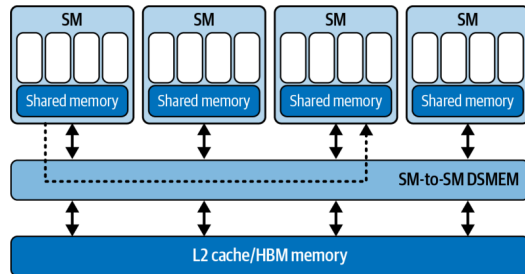


Figure 6-5. DSMEM shared across thread blocks in a cluster

Warps and SIMT: 32 Threads in Lockstep

- Blocks are subdivided into **warps of 32 threads** that execute one instruction together under **SIMT** (single instruction, multiple threads).
- The warp — not the thread — is what the **hardware actually schedules**; warp size has been **32 on every GPU generation**.
- The hardware hides long-latency events (global loads, cache fills, pipeline stalls) by **rapidly switching among warps**.

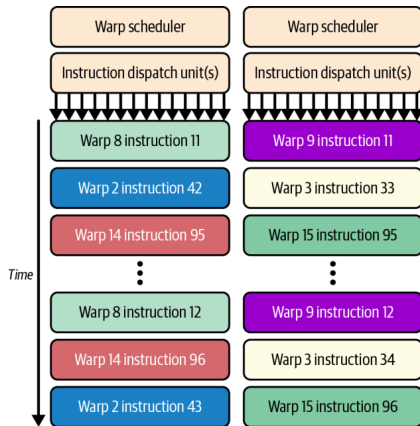


Figure 6-7. A warp advances as a whole under the warp scheduler

*Plain English: a warp is a **32-worker crew**: one instruction for all 32, everyone moves together — or the whole crew waits.*

Why SIMT Matters: One Instruction Drives 32 Lanes

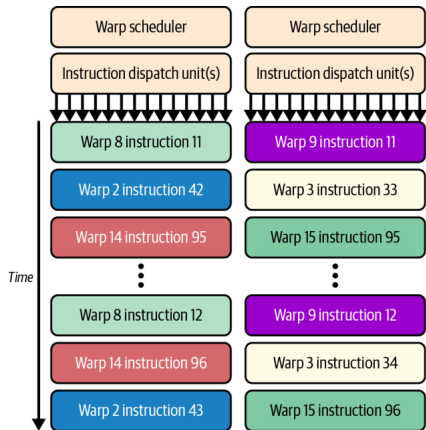


Figure 6-7. The warp advances as a whole under the warp scheduler

*Plain English: one work order commands **32 workers at once** — cheap to manage, powerful when they agree, awkward when they don't.*

- The scheduler issues **one instruction**; all **32 lanes** execute it on **32 different data elements**.
- Fetch / decode / schedule cost is paid **once per warp**, not once per thread — control logic shrinks, silicon goes to **ALUs**.
- This is why the **warp is the hardware's scheduling unit**: threads never advance alone; the crew moves together.
- The price of lockstep: if threads want **different paths**, the warp can't split — next page.

Warp Divergence: When if/else Splits the Crew

```
if (i % 2 == 0)
    taskA(); // 16 threads want this
else
    taskB(); // the other 16 want this
```

- A warp executes **one instruction at a time** — but here **16 lanes want A** and **16 want B**.
- So the warp runs **A with the B-threads masked**, then **B with the A-threads masked** — time $\approx$ **sum of both paths**.
- **No penalty across warps** — different warps branch freely.
- Fix: branch on **warp-aligned** conditions ($i / 32$), not per-thread parity ($i \% 2$).

*Plain English: if half the **crew** takes work order A and half takes B, the crew runs **each job twice**.*

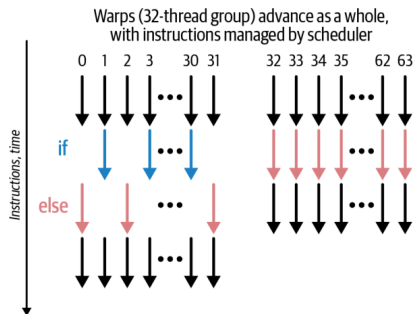


Figure 6-8. Divergence (left) vs. uniform execution (right)

Worked Example: One Million Adds Through Both Views

Software view: you write the order sheet

```
add<<<3907, 256>>>(A, B, C, N);
```

Grid	1 (this launch)
Blocks	3,907 = $\lceil 10^6 / 256 \rceil$
Threads / block	256 (your choice)
Warps / block	8 (= $256 \div 32$, HW slices)
Total	~1M threads, 31,256 warps

Each thread only says: "I handle element `idx`."

Why 256 threads per block? (best practice)

- **Multiple of 32**: blocks are sliced into warps — $256 = \mathbf{8 \text{ full warps}}$, no half-empty warp wasting a scheduler slot.
- **Well under the 1,024 cap**: each thread keeps enough **registers**, and the block's **shared-memory** share stays modest.
- **Small enough to stack**: an SM hosts **several 256-thread blocks** at once $\Rightarrow$ many resident warps $\Rightarrow$ **latency hiding**.
- Not magic: **128–512** are all reasonable — **profile** to pick.

- 4 SMs or 400 SMs: **same code** — the grid **decouples** the program from the GPU's size.

*Plain English: you write the **order sheet** (3,907 teams of 256 workers); the hardware assigns teams to workshops and slices each team into 32-worker crews.*

Let the Workshop Move: Latency Hiding, Cycle by Cycle

One partition, one moment

Warp A — **waiting** on an HBM load

Warp B — **waiting** on a previous result

Warp C — **ready** Warp D — **ready**

The scheduler skips A and B and **issues C or D** — skipping a stalled warp costs **nothing**.

Cycle	Issued (example)
1	Warp C
2	Warp F
3	Warp D
4	Warp H

*Plain English: latency hiding does **not** reduce waiting time — it **fills the waiting time** with other crews' work.*

- “Issued” = the instruction **starts**; it need not finish this cycle.
- Latency hiding does **not** shorten any warp's wait — it **fills the wait** with other warps' work.
- This is why **64 resident warps** matter: a deep pool of *ready* candidates for the 4 dispatchers — and why low occupancy hurts (nothing ready to switch to).

Occupancy Gives the Scheduler More Choices

Definition

Occupancy = active warps on an SM $\div$ the hardware maximum (64 on Blackwell).

“**Achieved occupancy**” is the measured average, reported by Nsight Compute.

- High occupancy $\Rightarrow$ when one warp stalls on memory, another is ready $\Rightarrow$ **latency hiding**.
- Blackwell's large register file (64K/SM) makes high occupancy easier to reach.
- The counterweight: warps share **registers and shared memory**. Pack in too many $\Rightarrow$ **register spilling** to slow memory — a new stall you created.
- So: profile occupancy **together with** register / shared-memory usage.

*Plain English: occupancy is **not** “how busy the GPU is” and **not** a performance score — it counts **how many resident crews the dispatcher can choose from**. Enough hides latency; beyond enough, more may not help (p. 52).*

Hardware Limits I: Warp and Block (Table 6-2)

No need to memorize these numbers. The one idea: each block's threads, registers, and shared memory decide how many blocks and warps still fit on one SM.

Resource	Limit (B200)
Warp size	32 threads
Max threads / block	1,024
Max warps / block	32 (= 1024 ÷ 32)

$$\text{blockDim.x} \times \text{blockDim.y} \times \text{blockDim.z} \leq 1024$$

Why multiples of 32 matter

A **33-thread block** occupies **two** warp slots — the second warp runs **1 of 32 lanes** but still consumes a full scheduler slot. Every non-multiple of 32 donates scheduler capacity to the void.

- Block too big $\Rightarrow$ too many registers per block $\Rightarrow$ **spilling**; or too much shared memory (227 KB usable is shared by **all resident blocks** on the SM).
- Smaller blocks can mean **higher occupancy** — more independent blocks fit per SM.

Hardware Limits II: SM-Resident and Grid (Tables 6-3 / 6-4)

Per SM (resident)

Limit

Max warps	64 (= 2,048 threads)
Max threads	2,048
Max blocks	32

Grid

Max blocks (X / Y / Z)	~2.1B / 65,535 / 65,535
Max concurrent grids	128 kernels per device

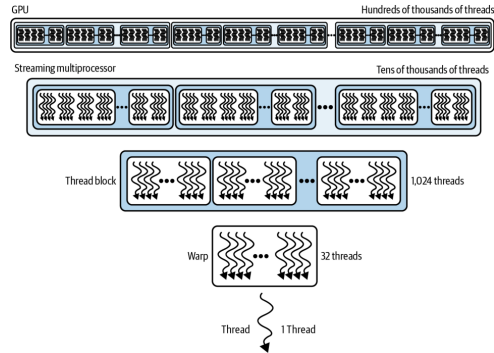


Figure 6-9. Relative scale of thread resources

- Worked example: 1024-thread blocks $\Rightarrow$ only **2 blocks/SM**; 256-thread blocks $\Rightarrow$ **8 blocks/SM** — same 2,048 threads, more flexibility.
- In practice you hit **per-SM limits** long before grid limits; $>65,535$ blocks in Y/Z $\Rightarrow$ use 2D/3D grids or multilaunch.^[1,2]

Compatibility: PTX, SASS, and Fatbins

- **SASS**: final machine code for *one* architecture (sm_90 = Hopper, sm_100 = Blackwell). SASS-only binaries **won't run on newer GPUs**.
- **PTX**: virtual ISA the driver **JIT-compiles** at load time $\Rightarrow$ **forward compatibility**.
- Family targets (sm_100f) restrict portability to the same feature family.

*Plain English: fast code must also **survive GPU generations** — ship optimized **SASS** for today's chips plus **PTX** for forward compatibility.*

Best practice

Ship a **fatbin**: generic PTX + the family-specific cubins you need for optimizations — with **fallback paths** for other architectures.

- Verify with `CUDA_FORCE_PTX_JIT=1`: forces PTX JIT; if the binary lacks PTX, **kernel launch fails** — rebuild with PTX.